

AI MEMORY / Mimir

Manual de Arquitetura e Implementação

Semantic Core / G_S

Auditoria integral da implementação atual, desenho do pipeline se-

mântico, contratos de dados, proveniência, retrieval híbrido, segurança, validação e roteiro incremental S0-S15.



Preparado a partir da análise da branch `main`, commit `f2995ca`, e da resposta técnica completa produzida sobre a evolução do repositório.

Edição estruturada em formato de instruções - 2026

Como usar este manual

Este documento preserva o conteúdo técnico da análise original e o reorganiza como um **manual de implementação**. A leitura recomendada depende do objetivo:

- Para entender o estado atual do repositório, leia as Partes I e II.
- Para começar a implementar imediatamente, vá às Partes III e VI.
- Para definir contratos e invariantes antes de codificar, concentre-se nas Partes III e V.
- Para memória de agentes e busca, use a Parte IV.
- Para auditoria, segurança e produção, use a Parte V.

Princípio diretor

O Semantic Core não deve substituir o que já existe. Ele deve herdar a filosofia do pipeline atual: **preservar, observar, derivar, validar e publicar**. A semântica entra como derivação versionada sobre uma camada estrutural verificável.

Conteúdo

Como usar este manual	i
1 Auditoria da implementação atual	1
1.1 Conclusão central da auditoria	1
1.2 O que o AI-MEMORY realmente é hoje	1
1.3 Estado por camada	1
1.4 A barreira semântica atual deve ser preservada	2
1.5 Pontos fortes da implementação existente	2
1.6 Limitações atuais relevantes ao Semantic Core	3
2 Arquitetura-alvo do Semantic Core	4
2.1 Introduza três grafos, não dois	4
2.1.1 Exemplo operacional	4
2.2 Defina o núcleo como ledger, não como índice	5
2.3 Pipeline completo recomendado	5
2.4 Absorva mecanismos modernos sem copiar um sistema inteiro	7
3 Identidade, evidência e contratos semânticos	8
3.1 Resolva a identidade longitudinal antes de embeddings	8
3.1.1 Não altere source.id	8
3.2 Use afirmações reificadas, não arestas simples	8
3.2.1 Contrato sugerido para Assertion	9
3.3 Modele o conhecimento como bitemporal	9
3.4 Separe tipos de memória	10
3.5 Implemente consolidação reversível	10
3.6 Crie provenance para inferência	11
3.7 Contratos mínimos recomendados	12
4 Retrieval, índices e memória de agentes	13
4.1 Não escolha entre vetor e grafo	13
4.2 Use decomposição para perguntas complexas	13
4.3 Mantenha os índices como projeções	13
4.4 Embeddings entram depois da identidade e dos contratos	14
4.5 Combine Dense, BM25, entidades, tempo e grafo	15
4.6 Working Memory para agentes	15
5 Multimodalidade, segurança e validação	17
5.1 OCR, áudio, vídeo e visão devem ser derivados	17
5.2 Entity Resolution deve ser reversível	17
5.3 Contradições são memória, não erro	18
5.4 Diferencie integridade de autenticidade	18
5.5 Isole parsers e extractors	19
5.6 Trate documentos como dados não confiáveis	19
5.7 Avalie cada subsistema separadamente	19

6	Roteiro de implementação S0-S15	21
6.1	Ordem recomendada	21
6.2	Primeiro incremento concreto: semantic-foundation	22
6.2.1	Quatro invariantes que esse incremento deve provar	22
6.3	Estrutura de código recomendada	22
6.4	Mudança posterior no Hot Hub	24
6.5	Checklist de implementação por pull request	24
7	Princípios que não devem ser violados	25
7.1	Evite transformar o projeto em um RAG genérico	25
7.2	Síntese arquitetural	25
7.3	Filosofia completa do pipeline	26
7.4	Ressalvas da auditoria	26
A	Referências técnicas e fontes de comparação	27
B	Mapa de aceitação do Semantic Core	28

Auditoria da implementação atual

1.1 Conclusão central da auditoria

A implementação atual já apresenta quase exatamente a fundação que deveria existir antes de se construir um Semantic Core sério. O passo correto não é transformar o BN1_2 em um banco vetorial, e sim construir acima dele um **ledger semântico temporal de afirmações e evidências**, mantendo G_P como a camada factual e estrutural imutável.

Decisão arquitetural

Não substitua a arquitetura existente. Estenda-a. Trate o AI-MEMORY como a camada de memória canônica e verificável; trate embeddings, grafos semânticos, resumos, entidades e índices como derivações reconstruíveis.

1.2 O que o AI-MEMORY realmente é hoje

O pipeline executável observado na branch `main` é:

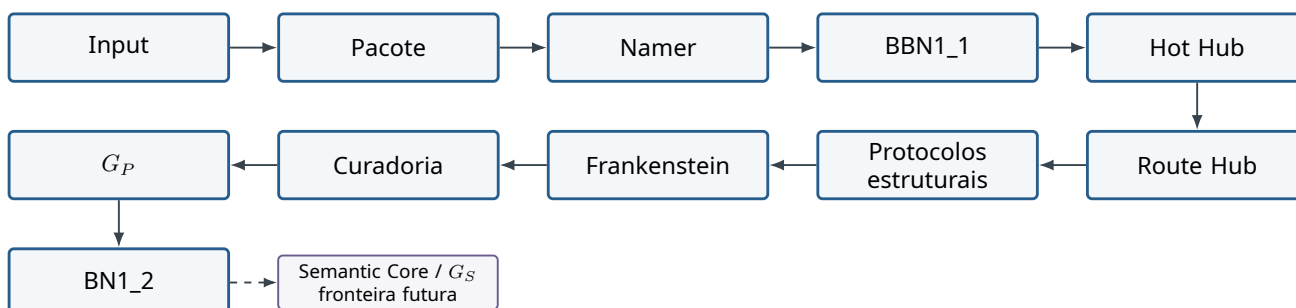


Figura 1.1: Pipeline executável atual e a fronteira em que o Semantic Core deve começar.

O próprio `README.md` da v0.2.0 declara que o pipeline termina em BN1_2; Semantic Core/G_S permanece como próxima etapa.

1.3 Estado por camada

Camada	Estado atual	Leitura arquitetural
Pacote	Ingestão de arquivos regulares, SHA-256 no recebimento, locking e recuperação de interrupções	Boa fundação de chain-of-custody.
Namer	Desacopla nome original da identificação operacional	Evita semântica acidental no início do pipeline.

Camada	Estado atual	Leitura arquitetural
BBN1_1	Buffer intermediário verificável	Correto como estágio transitório.
Hot_Hub	Representação universal lossless em chunks de 1024 bytes	Excelente camada canônica, ainda que custosa em JSON.
Route Hub	Roteamento pós-verificação por extensão	Interpretação vem depois da preservação.
Protocolos	TXT, JSON, CSV, DOCX, XLSX, PDF, imagem, WAV e mídia	Base multimodal suficiente para servir de substrato.
Frankenstein	Normaliza representações estruturais heterogêneas	Peça central para uniformização.
Curadoria	Impõe contrato estrutural estrito	Funciona como firewall epistemológico.
G_P	Apenas <code>derived_from</code> , <code>contains</code> , <code>precedes</code>	Mantém proveniência separada de significado.
BN1_2	Gerações imutáveis, verificadas e publicação atômica	Excelente fronteira de entrada do Semantic Core.
G_S	Não implementado	Deve ser uma derivação versionada, nunca substituir G_P .

1.4 A barreira semântica atual deve ser preservada

A Curadoria atual aceita apenas relações observáveis e rejeita qualquer relação semanticamente interpretativa. Em particular, o grafo de proveniência é limitado a:

```
derived_from
contains
precedes
```

Isso é uma propriedade incomum e valiosa. Relações como `explains`, `contradicts`, `means`, `supports` ou `causes` não devem ser acrescentadas a G_P .

Invariante I-01 - fronteira epistemológica

G_P contém somente fatos observáveis da transformação estrutural. Toda interpretação deve viver em uma camada superior, apontando de volta para evidências em G_P .

1.5 Pontos fortes da implementação existente

O repositório já possui três propriedades que são difíceis de acrescentar retrospectivamente:

1. **Gerações e imutabilidade:** o estado publicado é identificado e verificável.
2. **Proveniência forte:** objetos estruturais podem ser rastreados até sua origem.

3. **Separação entre observação e interpretação:** o pipeline não tenta transformar inferência em fato estrutural.

Também já existem mecanismos concretos de integridade: SHA-256, `O_NOFOLLOW`, validação de árvores, `fsync`, `locks`, `os.replace` atômico, `manifests` e verificação das gerações.

1.6 Limitações atuais relevantes ao Semantic Core

As principais limitações que devem orientar o próximo estágio são:

- A identidade estrutural é ligada à geração do Hot Hub e não é longitudinal.
- O mesmo conteúdo reingerido em outro ciclo recebe uma nova ocorrência.
- Não há OCR, ASR, embeddings, entity resolution, relações semânticas nem busca semântica.
- O pipeline ainda reconstrói o arquivo inteiro em memória.
- Parsers não são isolados por sandbox de CPU/memória.
- SHA-256 assegura integridade relativa ao inventário esperado, mas não autenticidade se dados e hashes forem alterados em conjunto.

Não confundir

A ausência atual de semântica não é um defeito da camada estrutural. É precisamente o que permite construir o Semantic Core sem contaminar a cadeia de evidência.

PARTE 2

Arquitetura-alvo do Semantic Core

2.1 Introduza três grafos, não dois

A arquitetura recomendada separa três níveis:

$$G_P \longrightarrow G_M \longrightarrow G_S$$

onde:

- G_P = grafo de proveniência estrutural observável;
- G_M = grafo de menções e observações semânticas extraídas;
- G_S = grafo de entidades, eventos, proposições e afirmações resolvidas.

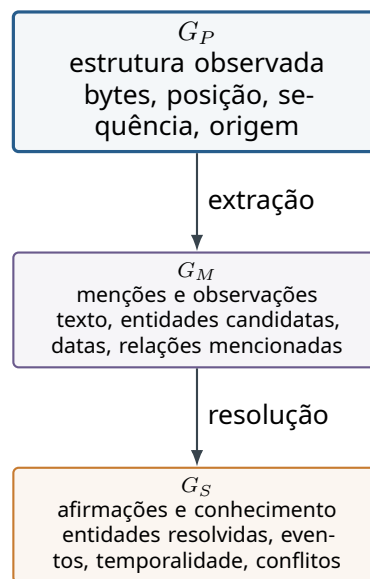


Figura 2.1: Separação entre evidência estrutural, menções extraídas e afirmações semânticas.

2.1.1 Exemplo operacional

Considere o texto:

"João entrou na OpenAI em 2023."

O sistema deve distinguir:

1. a sequência textual "João" em uma página - **menção**;
2. o mapeamento da menção para a entidade Joao Silva - **entity resolution**;
3. a proposição *worksAt*(João Silva, OpenAI) - **afirmação semântica**;

4. o ano 2023 como início de validade - **temporal resolution**.

```

G_P
|
+-- EvidenceAnchor
    |
    v
    G_M
    |-- Mention("Joao")
    |-- Mention("OpenAI")
    |-- DateMention("2023")
    `-- RelationMention("entrou em")
        |
        v
        Entity Resolution + Temporal Resolution
            |
            v
            G_S
            `-- Assertion:
                subject  = ent:joao_silva
                predicate = works_at
                object    = ent:openai
                valid_from ~ 2023
                evidence  = [...]
  
```

2.2 Defina o núcleo como ledger, não como índice

O núcleo recomendado é:

Evidence Ledger + Assertion Ledger + Temporal Semantics + Provenance

A relação fundamental é:

Canonical Memory $\neq$ Retrieval Index

Embeddings, BM25, índices de entidades, grafos de traversal, PageRank, community summaries e árvores hierárquicas devem ser **read models reconstruíveis**. O estado autoritativo deve continuar append-only ou versionado.

Padrão de arquitetura

A estrutura se aproxima de Event Sourcing + CQRS: gerações canônicas representam a verdade persistida; índices são projeções otimizadas para leitura.

2.3 Pipeline completo recomendado

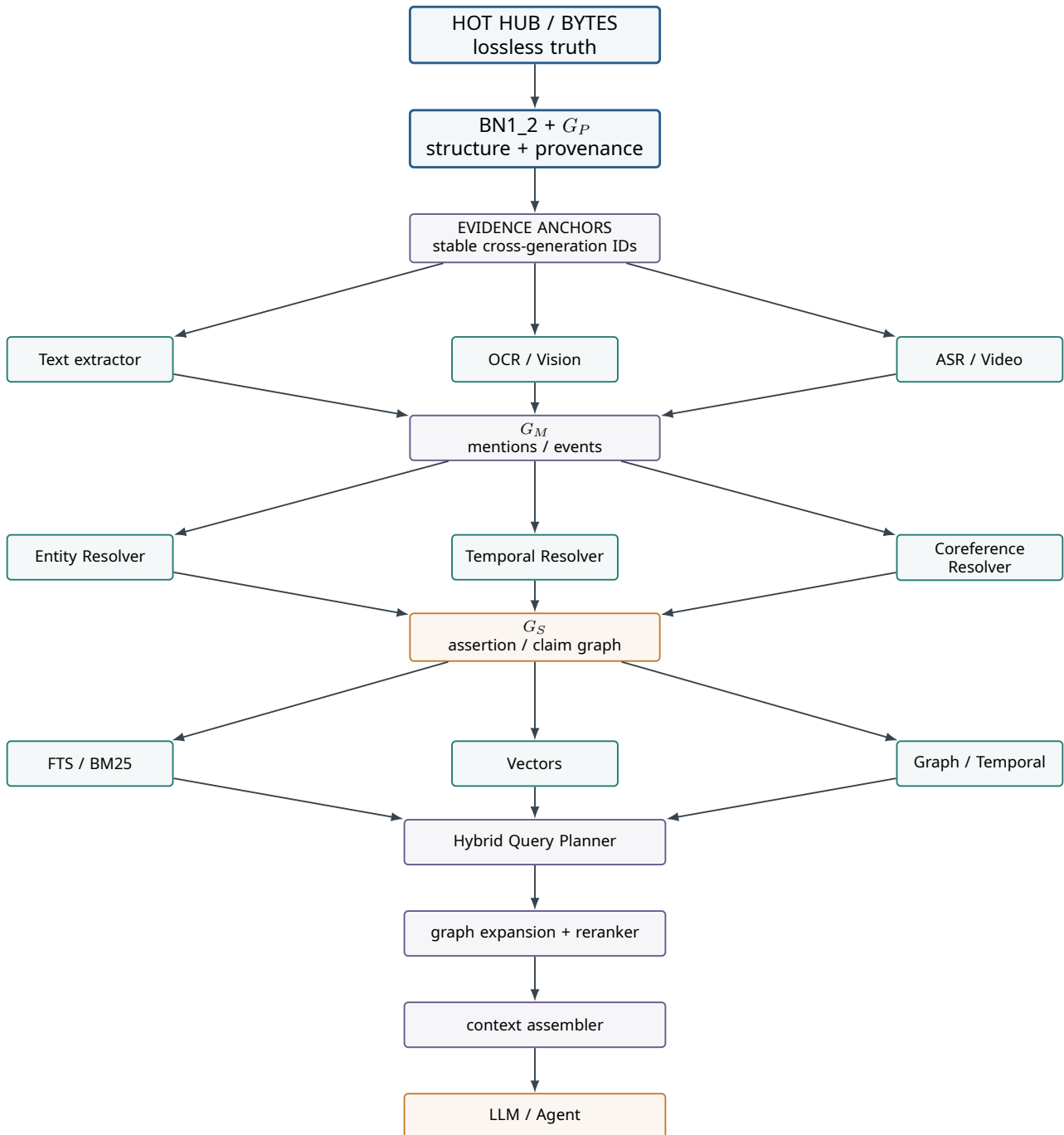


Figura 2.2: Arquitetura vetorial do Semantic Core proposto. Todos os índices abaixo de G_S são reconstruíveis.

2.4 Absorva mecanismos modernos sem copiar um sistema inteiro

A arquitetura híbrida recomendada combina mecanismos que aparecem separadamente em vários sistemas:

Sistema / trabalho	Mecanismo útil ao Mimir
MemGPT / Letta	Hierarquia entre memória residente e memória paginada; pequeno working set no contexto.
Mem0	Pipeline de extração, atualização, gerenciamento e recall; retrieval híbrido.
Zep / Graphiti	Knowledge graph temporal e tratamento explícito de knowledge updates.
HippoRAG	Recuperação associativa multi-hop com propagação/PageRank sobre grafo.
RAPTOR	Abstrações hierárquicas, clusters e resumos para perguntas em múltiplas escalas.
GraphRAG	Entidades, comunidades e respostas globais sobre coleções extensas.
Generative Agents	Episódios, reflexões e uso de memória na ação/planejamento.
A-MEM	Ligações dinâmicas entre memórias em estilo semelhante a Zettelkasten.
LongMemEval	Avaliação de memória longa, temporalidade, atualizações e abstention.
LoCoMo	Memória longitudinal em múltiplas sessões, temporal e causal.

Regra de integração

Nenhum desses mecanismos deve se tornar a fonte canônica. Eles são consumidores e projeções do ledger do Mimir.

PARTE 3

Identidade, evidência e contratos semânticos

3.1 Resolva a identidade longitudinal antes de embeddings

A identidade atual é intencionalmente ligada à geração:

```
source.id = hot_hub_generation + ":" + name
```

Isso responde muito bem a “qual objeto existia naquela geração?”, mas não responde sozinho a “eu já conhecia este mesmo conteúdo em outra época?”.

3.1.1 Não altere source.id

A identidade existente deve continuar representando a ocorrência histórica. Adicione identidades paralelas:

$$C(x) = SHA256(\text{bytes do arquivo})$$

$$A = H(C(x), kind, locator, protocol)$$

Sugestão de quatro IDs simultâneos:

```
source_occurrence_id = ocorrencia especifica da geracao
content_id            = identidade dos bytes
anchor_id             = identidade persistente da evidencia
structural_node_id    = objeto daquela transformacao especifica
```

Isso permite reconhecer que dois objetos estruturais de gerações diferentes apontam para o mesmo conteúdo, sem perder a proveniência de cada ocorrência.

Instrução S0-A

Implemente primeiro content_id e EvidenceAnchor. Não comece por embeddings. O objetivo inicial é criar continuidade longitudinal determinística.

3.2 Use afirmações reificadas, não arestas simples

Evite modelar diretamente:

```
Joao --works_at--> OpenAI
```

Uma relação semântica deve ser um objeto com identidade própria, porque ela pode ser contestada, temporal, negada, atualizada ou suportada por múltiplas evidências.

3.2.1 Contrato sugerido para Assertion

```
{
  "schema": "mimir.assertion.v1",
  "assertion_id": "assert:...",
  "proposition_id": "prop:...",
  "subject": "entity:joao",
  "predicate": "works_at",
  "object": "entity:openai",
  "polarity": "positive",
  "valid_time": {
    "from": "2023",
    "to": null,
    "precision": "year"
  },
  "evidence": [
    {
      "anchor_id": "anchor:...",
      "node_id": "obj:...",
      "property": "text",
      "char_start": 13,
      "char_end": 42
    }
  ],
  "epistemic_status": "reported",
  "inference": {
    "run_id": "run:...",
    "method": "semantic_extractor",
    "model": "...",
    "prompt_hash": "...",
    "code_version": "..."
  }
}
```

Uma mesma proposição pode ter múltiplas assertions:

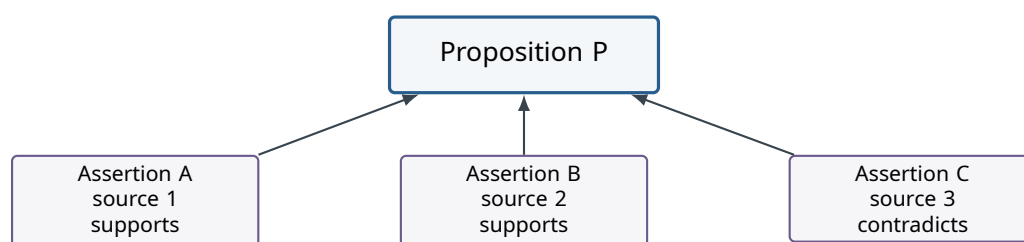


Figura 3.1: Reificação permite preservar suporte e contradição sem escolher prematuramente uma verdade.

3.3 Modele o conhecimento como bitemporal

O Semantic Core deve distinguir:

t_v = tempo em que algo era válido no mundo

t_t = tempo em que a memória tomou conhecimento

Exemplo: um documento recebido em 2026 declara que João trabalhou em uma empresa entre 2019 e 2022.

```
transaction_time = 2026-10-01
valid_from       = 2019
valid_to         = 2022
```

Isso permite responder corretamente a perguntas diferentes: estado atual, estado histórico, crença do sistema em uma data passada e momento em que uma informação foi corrigida.

Distinção crítica

O predicado estrutural precedes de G_P significa ordem de observação dentro do protocolo. Ele não representa tempo semântico entre eventos do mundo.

3.4 Separe tipos de memória

Depois de G_S , o sistema pode operar com vários tipos de memória:

Tipo	Papel
Source memory	Bytes originais no Hot Hub.
Structural memory	$BN1_2 + G_P$.
Episodic memory	Interações, eventos, execuções e experiências específicas.
Semantic memory	Entidades, relações, afirmações e conceitos consolidados.
Reflective memory	Resumos, abstrações, generalizações e regras derivadas.
Procedural memory	Procedimentos, estratégias e sequências de ação bem-sucedidas.
Working memory	Pequeno subconjunto mantido no contexto ativo do agente.

Invariante I-02 - consolidação não destrutiva

Reflexão nunca substitui episódio. Resumo, abstração e consolidação são derivados versionados; as memórias de origem permanecem recuperáveis.

3.5 Implemente consolidação reversível

Nunca faça:

```
memoria antiga
-> LLM resume
-> DELETE memoria antiga
```

Faça:

```
episodios E1 E2 E3 E4
|
```

```

v
reflection R1
|-- derived_from E1
|-- derived_from E2
|-- derived_from E3
`-- derived_from E4

```

Posteriormente:

```

R1 + R2 + R3
-> abstraction A1

```

Mantenha as dependências explícitas:

$$D(x) = \{y_1, y_2, \dots, y_n\}$$

Se uma dependência relevante for invalidada:

$$\Delta y_i \Rightarrow \text{invalidate}(x)$$

Assim, estruturas semelhantes a RAPTOR ou community reports podem ser regeneradas sem se tornarem fonte primária de verdade.

3.6 Crie provenance para inferência

Introduza um objeto InferenceRun com, no mínimo:

```

run_id
extractor_name
extractor_version
model_id
model_revision
prompt_template_hash
parameters
code_commit
input_generation
input_fingerprint
output_hash
timestamp

```

A cadeia desejada é:



Figura 3.2: Toda resposta auditável deve poder descer até a evidência estrutural e os bytes de origem.

Invariante I-03 - auditabilidade

Para toda afirmação persistente, deve existir um caminho verificável até a evidência de origem, salvo objetos explicitamente classificados como hipótese/reflexão. Mesmo estes precisam apontar para suas dependências.

3.7 Contratos mínimos recomendados

Crie inicialmente os seguintes schemas:

Schema	Responsabilidade
mimir.evidence-anchor.v1	Ponte estável entre BN1_2 e o Semantic Core.
mimir.mention.v1	Menção observada em evidência.
mimir.entity.v1	Entidade canônica resolvida.
mimir.event.v1	Evento semanticamente estruturado.
mimir.proposition.v1	Conteúdo lógico abstrato de uma proposição.
mimir.assertion.v1	Uma fonte ou extractor afirmando uma proposição.
mimir.inference-run.v1	Proveniência da inferência.
mimir.semantic-graph.v1	G_S .
mimir.semantic-generation.v1	Manifesto transacional da geração semântica.
mimir.index-profile.v1	Dependências e parâmetros de um índice derivado.

Crie um `semantic.validate(...)` tão estrito quanto o `curator.validate(...)` estrutural. Conceitualmente, isso se aproxima de validação por shapes: o grafo só é publicado se obedecer ao contrato definido.

PARTE 4

Retrieval, índices e memória de agentes

4.1 Não escolha entre vetor e grafo

O retrieval deve ser híbrido. Uma consulta pode exigir similaridade semântica, entidades, tempo, relações, ordem episódica e evidência ao mesmo tempo.

Tipo de pergunta	Retrieval dominante
Nome ou ID exato	FTS + entity index.
Pergunta factual local	BM25 + dense vector.
Conceito semanticamente semelhante	Dense retrieval.
Pergunta multi-hop	Graph expansion / Personalized PageRank.
Pergunta temporal	Interval filtering + query expansion.
"O que mudou?"	Comparação de assertions ao longo do tempo.
Pergunta global	Comunidades / RAPTOR / resumos hierárquicos.
Auditoria	Evidence-first; bypass de summaries.
"Como você sabe?"	Caminho $G_S \rightarrow evidence \rightarrow G_P \rightarrow Hot\ Hub$.

4.2 Use decomposição para perguntas complexas

O Query Planner deve conseguir decompor uma pergunta em subconsultas. O objetivo é evitar exigir que uma única busca encontre toda uma cadeia multi-hop implicitamente.

Exemplo:

```
Pergunta:
"Qual era o problema de isolamento antes de mudarmos a arquitetura?"

Subconsultas:
1. localizar o episodio da mudanca de arquitetura
2. recuperar memorias imediatamente anteriores
3. recuperar entidades/temas ligados a isolamento
4. expandir arestas do grafo semanticamente relacionadas
5. ordenar por transaction_time e valid_time
6. reranquear as evidencias finais
```

4.3 Mantenha os índices como projeções

Persistência canônica e tecnologia de busca devem permanecer desacopladas.

Componente	Escolha inicial recomendada
Catálogo local	SQLite.
Busca lexical	SQLite FTS5.
Entity index	SQLite.
Temporal index	SQLite.
Claim store	Gerações canônicas + catálogo SQLite.
Graph adjacency	SQLite inicialmente.
Vector index	Backend plugável.
Escala servidor	PostgreSQL + pgvector.
Vector especializado	Qdrant opcional.
Grafo avançado	Começar interno; evitar dependência obrigatória no início.

A interface de um vector backend pode ser deliberadamente pequena:

```
class VectorIndex:
    def build(self, generation, records): ...
    def search(self, query, k): ...
    def verify_profile(self): ...
```

Todo índice deve registrar seu perfil:

```
semantic_generation
embedding_model
embedding_model_revision
dimension
normalization
chunking_profile
index_algorithm
index_version
```

Se o perfil mudar, o índice deve ser reconstruído. Nunca permita que uma mudança de índice modifique a memória canônica.

4.4 Embeddings entram depois da identidade e dos contratos

Embeddings devem apontar para objetos semânticos ou Evidence Anchors, nunca definir a identidade desses objetos.

$$e_i = E_{\theta}(x_i)$$

Se o modelo muda:

$$E_{\theta_1}(x) \neq E_{\theta_2}(x)$$

embora a memória x seja a mesma. Portanto:

$embedding \neq memoria$
 $embedding = indice\ de\ recuperacao\ de\ uma\ memoria$

4.5 Combine Dense, BM25, entidades, tempo e grafo

O conjunto de candidatos pode ser definido como:

$$C(q) = C_{BM25} \cup C_{dense} \cup C_{entity} \cup C_{temporal} \cup C_{graph}$$

Uma fusão inicial robusta é Reciprocal Rank Fusion:

$$RRF(d) = \sum_i \frac{1}{k + r_i(d)}$$

Depois, um reranker pode usar features:

$$S(d, q) = w_1 S_{semantic} + w_2 S_{lexical} + w_3 S_{entity} + w_4 S_{time} + w_5 S_{graph} + w_6 S_{evidence}$$

Para documentos técnicos, uma etapa de late interaction em granularidade de tokens também pode ser útil, pois evita reduzir toda uma passagem a um único vetor.

4.6 Working Memory para agentes

A memória longa não deve ser despejada integralmente no contexto do LLM. Mantenha um working set pequeno e estável:

Core Working Memory

identity
current objective
active project
current constraints
recent commitments
open problems
critical entities

A divisão recomendada é:

Semantic Core = memoria longa autoritativa
Context Pack = working set temporario
LLM context = cache extremamente curto

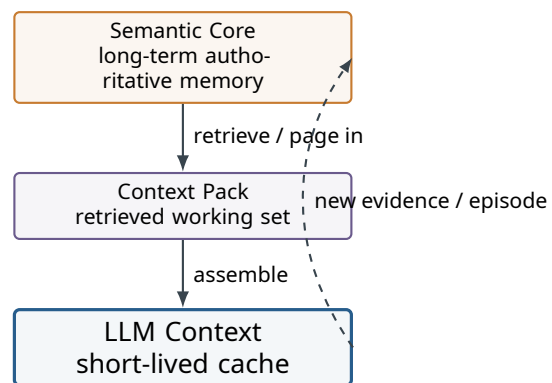


Figura 4.1: Hierarquia recomendada para agentes persistentes.

PARTE 5

Multimodalidade, segurança e validação

5.1 OCR, áudio, vídeo e visão devem ser derivados

O BN1_2 já observa formatos multimodais estruturalmente. A compreensão semântica deve ser acrescentada depois.

```
image node
-> OCR
  |-- text
  |-- bounding box
  `-- confidence

audio stream
-> ASR
  |-- text
  |-- sample_start
  |-- sample_end
  `-- speaker candidate

video
-> ASR + keyframes + vision
  |-- events
  |-- objects
  |-- speech
  `-- temporal segments
```

Um OCR não deve alterar o nó de imagem. Deve criar uma nova observação semântica que aponta para ele. Assim, o sistema preserva a diferença entre “estes pixels existem” e “um OCR interpretou estes pixels como determinada palavra”.

5.2 Entity Resolution deve ser reversível

Não substitua todas as strings parecidas por uma entidade de forma destrutiva. Modele menções separadas e resoluções versionadas:

```
Mention M1 --candidate_member_of--> Entity E1
Mention M2 --candidate_member_of--> Entity E1
Mention M3 --candidate_member_of--> Entity E1
```

Associe à resolução:

```
score
method
features
generation
```

```
human_override
```

Se uma menção tiver sido resolvida incorretamente, publique uma nova resolução. Não reescreva a menção original.

Invariante I-04

Observação imutável; interpretação versionada.

5.3 Contradições são memória, não erro

Quando duas fontes divergem, preserve ambas:

```
A1: "X nasceu em 1992" -- source A
A2: "X nasceu em 1993" -- source B
A1 --conflicts_with--> A2
```

Uma visão corrente pode calcular posteriormente uma crença:

$$\text{belief}(P) = f(\text{support}, \text{source diversity}, \text{recency}, \text{quality}, \text{contradictions})$$

Mas $\text{belief}(P)$ é uma projeção, não o registro histórico. O agente deve conseguir dizer “há fontes conflitantes” em vez de apagar silenciosamente uma delas.

5.4 Diferencie integridade de autenticidade

A implementação atual protege bem contra muitas corrupções e alterações locais inesperadas. Porém, se um atacante puder modificar dados e os hashes esperados, SHA-256 sozinho não autentica a origem.

Para uma versão de alta garantia, adicione opcionalmente manifests assinados, por exemplo com Ed25519:

$$\text{signature} = \text{Sign}_{sk}(H(\text{manifest}))$$

$$\text{Verify}_{pk}(\text{signature}, H(\text{manifest}))$$

Outra opção é encadear gerações:

$$H_n = H(\text{manifest}_n, H_{n-1})$$

Isso cria uma cadeia tamper-evident. Não precisa ser obrigatório na primeira versão do Semantic Core, mas deve ser previsto no contrato.

5.5 Isole parsers e extractors

Os limites atuais de bytes, nós, texto expandido e pixels são úteis, mas não equivalem a sandbox. Para arquivos arbitrários, execute parsers, OCR, ffmpeg/decoders e modelos em workers isolados.

```
parser / OCR / ffmpeg / image decoder / model
      |
      v
isolated worker
```

O worker deve receber, idealmente:

- filesystem read-only;
- rede desabilitada;
- RLIMIT_AS / limite de memória;
- RLIMIT_CPU;
- timeout externo;
- seccomp ou container opcional;
- limite máximo de output;
- IPC estruturado;
- validação do output no processo principal.

5.6 Trate documentos como dados não confiáveis

Quando a memória começar a conter conteúdo textual arbitrário, uma frase do documento como “ignore todas as instruções anteriores” deve continuar sendo dado, não instrução.

```
SYSTEM:
extract facts according to schema

DATA:
<untrusted_document>
...
</untrusted_document>
```

Nenhuma ação, ferramenta, URL ou código presente numa memória deve ser executado automaticamente durante extração. A saída deve passar por schema validation antes de ser publicada.

5.7 Avalie cada subsistema separadamente

Não avalie apenas a resposta final do agente. Meça cada componente:

Subsistema	Métrica recomendada
Structural -> Evidence	100% de anchors resolvíveis.
Extraction	Precision / Recall / F1.
Entity Resolution	Pairwise F1.
Temporal Resolution	Exact match + interval overlap.
Retrieval	Recall@K / MRR / nDCG.
Multi-hop	Evidence-chain recall.
Current state	Stale-fact error rate.
Contradições	Conflict detection F1.
Abstention	Precision/recall de “não sei”.
Provenance	100% das claims com cadeia resolvível.
Consolidation	Delta de performance antes/depois.
Index rebuild	Equivalência de resultados.
Latência	p50 / p95.
Custos de LLM	Tokens por write/read.
Robustez	Prompt injection, parser fuzz, geração corrompida.

A propriedade central de auditoria pode ser expressa como:

$$\boxed{\forall c \in G_S, \exists path(c, G_P, source)}$$

PARTE 6

Roteiro de implementação S0–S15

6.1 Ordem recomendada

Não comece por LLM, embeddings ou GraphRAG. Construa primeiro identidade, geração semântica e contratos determinísticos.

Etapa	Implementação	Resultado esperado
S0	EvidenceAnchor, content_id, identidade cross-generation	Continuidade longitudinal determinística.
S1	Manifesto e gerações Semantic_Core com upstream fingerprint do BN1_2	Fundação transacional.
S2	Mention, Entity, Event, Proposition, Assertion, InferenceRun	Ontologia operacional mínima.
S3	Validator semântico rígido	Semantic firewall.
S4	SQLite catalog + FTS5	Busca real sem depender de LLM.
S5	Extractor semântico plugável com saída estruturada	$BN1_2 \rightarrow G_M$.
S6	Entity resolution + temporal normalization	$G_M \rightarrow G_S$.
S7	Contradição, supersession e bitemporalidade	Knowledge updates corretos.
S8	Embedding interface + vector read model	Similaridade semântica.
S9	Hybrid retrieval + planner + fusion + reranker	Recall robusto.
S10	Graph traversal / PPR	Memória associativa multi-hop.
S11	Episodic memory + working memory	Memória persistente para agente.
S12	Reflection / consolidation / RAPTOR	Abstração hierárquica sem apagar episódios.
S13	OCR / ASR / Vision	Multimodalidade semântica.
S14	ACL, privacy, GC, signatures	Preparação para produção e alta garantia.
S15	LongMemEval / LoCoMo harness	Auditoria quantitativa fim a fim.

6.2 Primeiro incremento concreto: semantic-foundation

Se a próxima branch fosse criada agora, o primeiro incremento não deveria se chamar add-embeddings. Um nome coerente seria:

```
semantic-foundation
```

Estrutura inicial:

```
Transformer_Core/
|-- Hot_Hub/
|-- structural/
`-- semantic/
    |-- model.py
    |-- evidence.py
    |-- validation.py
    |-- pipeline.py
    |-- extractors/
    |-- resolution/
    |-- graph/
    |-- indexes/
    |-- retrieval/
    `-- consolidation/

Semantic_Core/
|-- manifest.json
|-- generations/
|   |-- <generation>/
|       |-- entities.jsonl
|       |-- mentions.jsonl
|       |-- assertions.jsonl
|       |-- events.jsonl
|       `-- inference_runs.jsonl
|-- indexes/
    `-- <semantic_generation>/<index_profile>/
```

6.2.1 Quatro invariantes que esse incremento deve provar

1. Mesmos bytes em ciclos diferentes produzem o mesmo content_id.
2. A mesma evidência estrutural produz o mesmo stable anchor.
3. Representações estruturais diferentes continuam sendo ocorrências diferentes e rastreáveis.
4. Uma geração semântica depende exatamente de uma geração BN1_2 verificada.

Critério de conclusão de S0/S1

O projeto deixa de ser apenas um pipeline que prepara arquivos para semântica e passa a ter uma identidade longitudinal explícita, verificável e independente do modelo de embedding.

6.3 Estrutura de código recomendada

A continuação natural de Transformer_Core/structural é:

```
Transformer_Core/
|-- Hot_Hub/
|-- structural/
|   |-- model.py
|   |-- protocols.py
|   |-- curator.py
|   `-- pipeline.py
`-- semantic/
    |-- model.py
    |-- evidence.py
    |-- pipeline.py
    |-- validation.py
    |
    |-- extractors/
    |   |-- text.py
    |   |-- llm.py
    |   |-- ocr.py
    |   |-- audio.py
    |   `-- vision.py
    |
    |-- resolution/
    |   |-- entities.py
    |   |-- coreference.py
    |   `-- temporal.py
    |
    |-- graph/
    |   |-- mentions.py
    |   |-- assertions.py
    |   |-- contradictions.py
    |   `-- traversal.py
    |
    |-- indexes/
    |   |-- lexical.py
    |   |-- vector.py
    |   |-- entity.py
    |   `-- temporal.py
    |
    |-- retrieval/
    |   |-- planner.py
    |   |-- fusion.py
    |   |-- rerank.py
    |   `-- context.py
    |
    `-- consolidation/
        |-- episodes.py
        |-- reflection.py
        `-- hierarchy.py
```

Não use o nome BN1_3 sem confirmar que ele já não possui significado específico no mapa arquitetural do projeto.

6.4 Mudança posterior no Hot Hub

A representação atual com arrays JSON é excelente para auditabilidade, mas cara em espaço. Um byte de origem pode consumir vários caracteres ASCII no JSON.

Não mude isso enquanto o Semantic Core está nascendo. Depois, considere uma `mimir.byte-chunks.v2` com:

```
raw chunk blob
+
JSON metadata
```

ou outro container binário versionado. Também migre a reconstrução para streaming:

```
read chunk -> process -> discard
```

em vez de:

```
vector = reconstruct_entire_file()
```

Isso reduz uma das limitações mais claras do pipeline atual sem misturar a mudança com o nascimento do Semantic Core.

6.5 Checklist de implementação por pull request

Para cada PR semântico, use o seguinte checklist:

1. Definir o schema e sua versão.
2. Definir a origem verificável de cada campo persistido.
3. Classificar campos como observados, inferidos ou calculados.
4. Definir identidade determinística e regras de colisão.
5. Definir upstream fingerprint e dependências de perfil.
6. Implementar validator antes da publicação.
7. Garantir publicação atômica e preservação da geração anterior em falha.
8. Criar testes de corrupção, path traversal, symlink, output parcial e mudança de upstream.
9. Criar teste de reprocessamento idempotente.
10. Criar teste de reversibilidade: apagar índices derivados e reconstruí-los sem perda da memória canônica.
11. Criar teste de auditabilidade: cada assertion deve resolver até EvidenceAnchor, nó estrutural, fonte e bytes.
12. Só então acrescentar otimizações ou modelos probabilísticos.

Princípios que não devem ser violados

7.1 Evite transformar o projeto em um RAG genérico

A principal armadilha seria reduzir o AI-MEMORY ao pipeline comum:

```
documento -> splitter -> embedding -> vector DB
```

O diferencial do projeto é:

```
bytes -> estrutura -> proveniencia -> semantica
```

A segunda arquitetura permite auditar de onde uma crença veio, qual parser a produziu, qual modelo a inferiu, qual versão existia naquele momento, qual evidência a contradizia, quando ela deixou de ser atual e quais bytes sustentavam toda a cadeia.

7.2 Síntese arquitetural

A definição compacta do alvo é:

Mimir Semantic Core = immutable evidence
+ versioned assertions
+ bitemporal knowledge graph
+ hybrid associative retrieval
+ non-destructive consolidation

E o ponto de conexão correto é:

$$BN1_2 \longrightarrow \textit{Semantic Core}$$

Não:

- Hot Hub -> embeddings;
- PDF -> LLM;
- G_P -> grafo semântico sobrescrito.

Regra final de arquitetura

BN1_2 funciona como a fronteira de evidência do sistema inteiro. G_S é uma interpretação versionada dessa evidência, nunca sua substituição.

7.3 Filosofia completa do pipeline

O pipeline atual segue:

preserve → observe → derive → validate → publish

O Semantic Core deve estender essa sequência para:

evidence → mentions → entities/events → assertions → temporal graph → retrieval → reflection

com todas as setas rastreáveis para trás.

7.4 Ressalvas da auditoria

A análise técnica cobriu diretamente os contratos e o código em:

- Transformer_Core/Hot_Hub/hub.py;
- Transformer_Core/structural/model.py;
- Transformer_Core/structural/protocols.py;
- Transformer_Core/structural/curator.py;
- Transformer_Core/structural/pipeline.py;
- BN1_2/buffer.py;
- BN1_1/Pacote/cache.py;
- testes, README, changelog, CLI e workflow de CI.

A referência visual interna docs/images/mimir-pipeline-2026-09-28.jpg não pôde ser auditada visualmente pelo conector GitHub durante a análise, pois o binário não foi entregue como uma imagem decodificável. Isso não afetou a reconstrução do pipeline executável, derivada diretamente do código e dos contratos.

Também não foi afirmado que o commit de merge auditado tinha CI verde: o workflow estava configurado para Python 3.10 e 3.12, mas o conector não retornou workflow run/status associado ao commit naquele momento.

Referências técnicas e fontes de comparação

As referências abaixo foram utilizadas como pontos de comparação arquitetural na análise original. Elas não substituem os contratos do próprio repositório.

- MemGPT: <https://arxiv.org/abs/2310.08560>
- Generative Agents: <https://arxiv.org/abs/2304.03442>
- HippoRAG: <https://arxiv.org/abs/2405.14831>
- RAPTOR: <https://arxiv.org/abs/2401.18059>
- A-MEM: <https://arxiv.org/abs/2502.12110>
- LongMemEval: <https://arxiv.org/abs/2410.10813>
- W3C RDF 1.2 Concepts: <https://www.w3.org/TR/rdf12-concepts/>
- W3C PROV Constraints: <https://www.w3.org/TR/prov-constraints/>
- W3C SHACL: <https://www.w3.org/TR/shacl/>
- SQLite FTS5: <https://sqlite.org/fts5.html>
- ColBERT: <https://arxiv.org/abs/2004.12832>
- sqlite-vec: <https://github.com/asg017/sqlite-vec>
- Mem0 migration / architecture material: <https://github.com/mem0ai/mem0>

PARTE B

Mapa de aceitação do Semantic Core

Considere o núcleo pronto para servir de base a agentes apenas quando os seguintes critérios forem simultaneamente verdadeiros:

1. Todo objeto semântico publicado pertence a uma geração semanticamente identificada.
2. Toda geração semântica aponta para exatamente um upstream BN1_2 verificado.
3. Toda assertion possui evidência ou é explicitamente marcada como hipótese/reflexão derivada.
4. Toda evidência resolve até um nó estrutural e, quando aplicável, até bytes de origem.
5. Reingerir os mesmos bytes em outro ciclo preserva o `content_id`.
6. Contradições não apagam assertions anteriores.
7. Atualizações temporais não reescrevem silenciosamente o histórico.
8. Um índice vetorial completo pode ser apagado e reconstruído sem mudar a memória canônica.
9. Mudanças de modelo, prompt ou versão de extractor criam novo perfil de inferência.
10. Falha durante publicação preserva a última geração válida.
11. Entradas malformadas, parsers ausentes e limites excedidos não geram conhecimento parcialmente publicado como completo.
12. Documentos não confiáveis não conseguem transformar texto em instrução executável durante extração.
13. O sistema consegue responder “como você sabe?” percorrendo a cadeia de proveniência.
14. Benchmarks medem retrieval, temporalidade, atualização, contradição, abstention e auditabilidade separadamente.

Estado-alvo

Quando esses critérios forem satisfeitos, o AI-MEMORY deixa de ser apenas um pipeline de ingestão e passa a funcionar como um **substrato de memória auditável para agentes**: preserva a evidência, registra interpretações versionadas, suporta atualização temporal e oferece retrieval híbrido sem confundir índice com verdade.

Documento estruturado e vetorial. Diagramas, linhas, tipografia e tabelas são gerados como objetos vetoriais no PDF; não foram rasterizados.